WEEK 5 – ERROR DETECTION

Hamming (7,4) Implementation and Instruction-by-Instruction Test Cases

Project	MicroOS-Sim – Educational Microcontroller Simulator
Team	Onji Byte
Members	Hisham, Izhan, Keora and Punarvi

1. Introduction

This report explains the Week 5 Error Detection update using the Hamming (7,4) method. The simulator calculates parity bits, checks the received codeword using syndrome bits and displays the result through an ALERT popup. Both test cases are explained step by step: one with an intentionally introduced error and one without an error.

2. Hamming (7,4) Overview

Hamming (7,4) uses four data bits and three even-parity bits. The seven bit positions are arranged as follows:

Position	1	2	3	4	5	6	7
Bit	P1	P2	D1	P4	D2	D3	D4

Parity equations

```
P1 = D1 XOR D2 XOR D4
P2 = D1 XOR D3 XOR D4
P4 = D2 XOR D3 XOR D4
```

The syndrome bits are calculated by checking the parity groups. The final syndrome is read as S4 S2 S1. A value of 000 means no error is detected. A non-zero value indicates the bit position of a single-bit error.

3. Common Input and Parity Calculation

Both programs use the same input data: D1=1, D2=0, D3=1 and D4=1. The parity values are P1=0, P2=1 and P4=0. Therefore, the original 7-bit codeword is **0110011**.

Instruction	Explanation
MOV R0,#01H	Stores D1=1 in R0.
MOV R1,#00H	Stores D2=0 in R1.
MOV R2,#01H	Stores D3=1 in R2.
MOV R3,#01H	Stores D4=1 in R3.
MOV A,R0; XRL A,R1; XRL A,R3; MOV R4,A	Calculates P1 = D1 XOR D2 XOR D4 = 0 and stores it in R4.
MOV A,R0; XRL A,R2; XRL A,R3; MOV R5,A	Calculates P2 = D1 XOR D3 XOR D4 = 1 and stores it in R5.
MOV A,R1; XRL A,R2; XRL A,R3; MOV R6,A	Calculates P4 = D2 XOR D3 XOR D4 = 0 and stores it in R6.

4. Case 1 – Error Detection (Hamming74Demo.asm)

In this case, the program intentionally changes the data bit at Hamming position 3. Since position 3 contains D1, the bit is flipped from 1 to 0. The received codeword becomes **0100011**.

Instruction	Explanation
MOV A,R0	Copies the original D1 value (1) into accumulator A.
XRL A,#01H	XORs A with 1. This flips the bit from 1 to 0.
MOV R0,A	Stores the changed D1 value (0) back in R0.
MOV A,R4; XRL A,R0; XRL A,R1; XRL A,R3; MOV R7,A	Calculates S1 using P1 and received D1, D2 and D4. Result: S1=1.
MOV A,R5; XRL A,R0; XRL A,R2; XRL A,R3; MOV R4,A	Calculates S2 using P2 and received D1, D3 and D4. Result: S2=1.
MOV A,R6; XRL A,R1; XRL A,R2; XRL A,R3; MOV R5,A	Calculates S4 using P4 and received D2, D3 and D4. Result: S4=0.

Final syndrome: S4S2S1 = 0 1 1 = **011**. Binary 011 equals decimal 3, so the error is identified at Hamming bit position 3.

UI Alert: Error detected at Hamming bit position 3.

5. Case 2 – No Error Detection (Hamming74NoError.asm)

In this case, the program performs the same input loading and parity calculation, but it does not flip or change any bit. The received codeword remains **0110011**.

Instruction	Explanation
MOV R0,#01H; MOV R1,#00H; MOV R2,#01H; MOV R3,#01H	Loads D1=1, D2=0, D3=1 and D4=1 into the registers.
MOV A,R0; XRL A,R1; XRL A,R3; MOV R4,A	Calculates P1=0.
MOV A,R0; XRL A,R2; XRL A,R3; MOV R5,A	Calculates P2=1.
MOV A,R1; XRL A,R2; XRL A,R3; MOV R6,A	Calculates P4=0.
MOV A,R4; XRL A,R0; XRL A,R1; XRL A,R3; MOV R7,A	Checks the first parity group. Result: S1=0.
MOV A,R5; XRL A,R0; XRL A,R2; XRL A,R3; MOV R4,A	Checks the second parity group. Result: S2=0.
MOV A,R6; XRL A,R1; XRL A,R2; XRL A,R3; MOV R5,A	Checks the fourth parity group. Result: S4=0.

Final syndrome: S4S2S1 = 0 0 0 = **000**. Therefore, no error is detected.

UI Alert: No error detected (syndrome 000).

6. Comparison of Both Test Cases

Details	Case 1 – Error	Case 2 – No error
Program	Hamming74Demo.asm	Hamming74NoError.asm
Input data	1, 0, 1, 1	1, 0, 1, 1
Original codeword	0110011	0110011
Bit modification	Position 3 is flipped	No bit is changed
Received codeword	0100011	0110011
Syndrome bits	S1=1, S2=1, S4=0	S1=0, S2=0, S4=0
Final syndrome	011	000
UI alert	Error detected at Hamming bit position 3.	No error detected (syndrome 000).

7. Java and UI Support

The assembly program uses MOV for data transfer and XRL for XOR calculations. Java instruction support was updated to execute XRL and ALERT. ALERT sends the result from the Core Process to the UI, where a popup displays the appropriate message for both test cases.

8. Team Members and Responsibilities

Member	Responsibilities
Hisham	Worked on the Hamming assembly algorithm, parity generation and test program integration. Tested the output of both error and no-error cases.
Izhan	Worked on Java instruction support for XRL and ALERT and related testing.
Keora	Worked on syndrome calculation, error-position identification and test verification.
Punarvi	Worked on UI alert integration, displaying the result in the interface and README documentation.

9. Running the Tests

The test programs are stored in **src/week4/hamming/**. Start the MicroOS-Sim processes, open the UI, load either Hamming74Demo.asm or Hamming74NoError.asm, and use RUN to execute the complete program or STEP to execute one instruction at a time. The corresponding alert is displayed in the UI.

10. Limitation and Conclusion

The current implementation detects and identifies a single-bit error. It does not automatically correct the incorrect bit. The two test cases demonstrate both outcomes: syndrome 011 produces an error-position alert, while syndrome 000 produces a no-error alert. This update adds Hamming (7,4) error detection to MicroOS-Sim while retaining the existing simulator functionality.